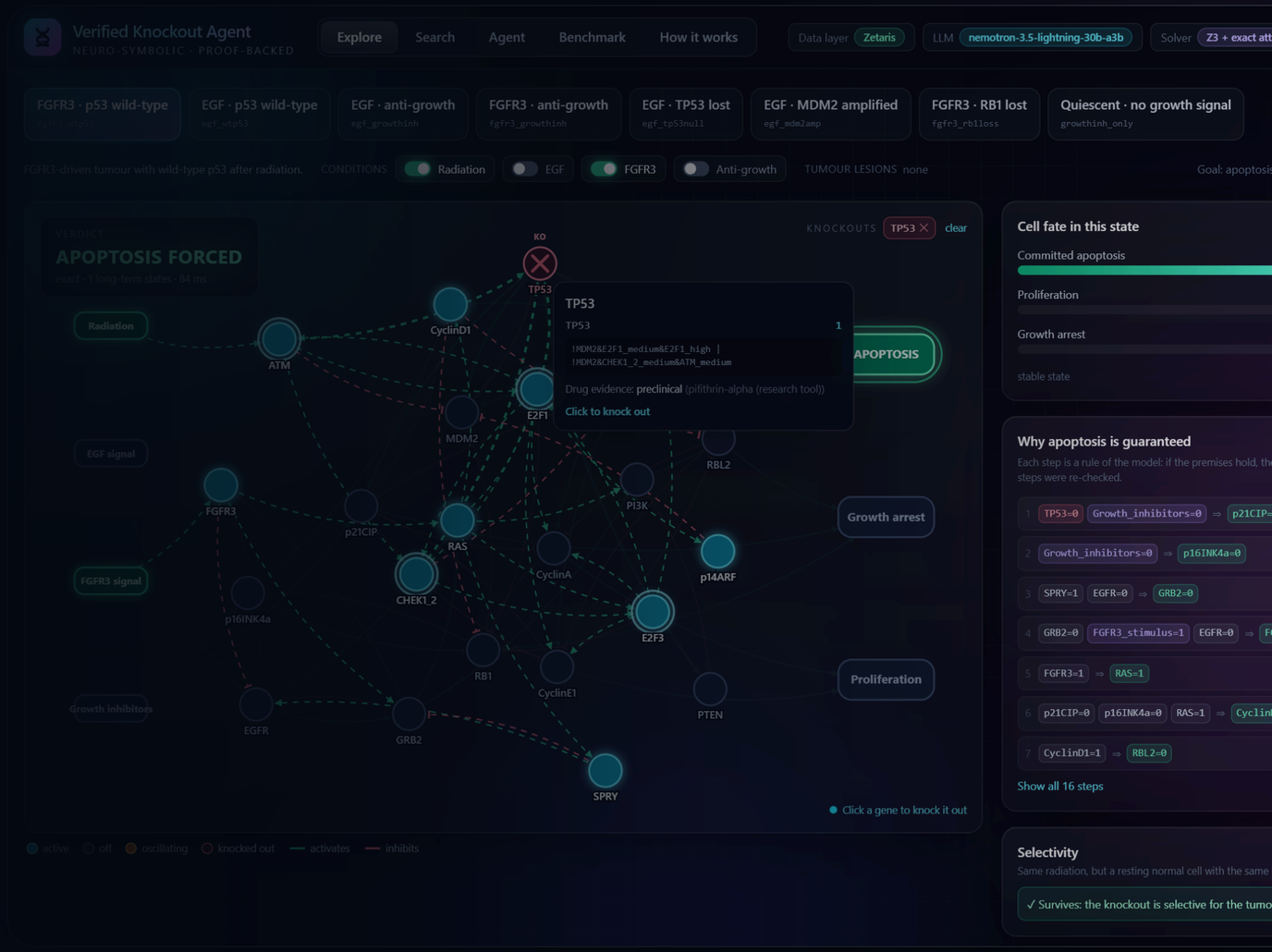
OPEN AGENT HACKATHON 2026 · TRACK 02

THE AGENT THAT CAN EXPLAIN WHY

Verified Knockout Agent

An LLM proposes.
An exact solver proves.
The explanation is checked.



Andreas Petros Rigoutsos · team “cache me if u can”

Nemotron · Zetaris · Meterless H-MEM

THE PROBLEM

A tumour cell can survive radiation. An LLM will happily tell you how to stop it, right or wrong

After radiation, a tumour with working p53 stops its cell cycle and repairs instead of dying.

Which genes should we switch off so the cell commits to **apoptosis**? The combinations explode and every wrong guess is a wasted experiment.

WHO IT IS FOR

Computational oncology and radiobiology groups choosing which knockouts to test, target-discovery teams that need a ranked and explained shortlist, and reviewers who must be able to check the reasoning.

We measured what an LLM does when asked, with no checker behind it:

91%

of its knockout proposals are wrong

invents

a solution for the one scenario where none exists

Nemotron 3.5, 8 scenarios, exhaustive ground truth

Published model: Remy et al., Cancer Research 2015 (35 genes)

THE IDEA

Propose, prove, explain: every claim goes through the solver

Zetaris

network rules, scenarios, drug evidence over SQL REST. The LLM writes its own `run_sql`

→

Nemotron

via Meterless Nightshift: proposes knockouts, writes SQL, words the explanation

→

Exact solver

every long-term state, including oscillations (Z3 + attractors). Verdict or a counterexample

→

Complete search

finds every minimal set or proves that none exists

→

Checked explanation

proof steps re-checked against the rules; stray claims are flagged

↺ **Counterexample feedback**

the surviving state names the genes that keep the cell alive; they go back to the LLM and into the search

Official Meterless H-MEM memory

store · ranked retrieval · feedback deltas · audit ledger. Conformance suite 13/13

WHY IT WORKS

A failure tells us what to try next, so the search cannot miss

1 · A knockout fails

The solver returns the surviving state and the genes **active** in it.

2 · Only those genes can break it

Adding a gene that is already off changes nothing.

3 · So extend only with them

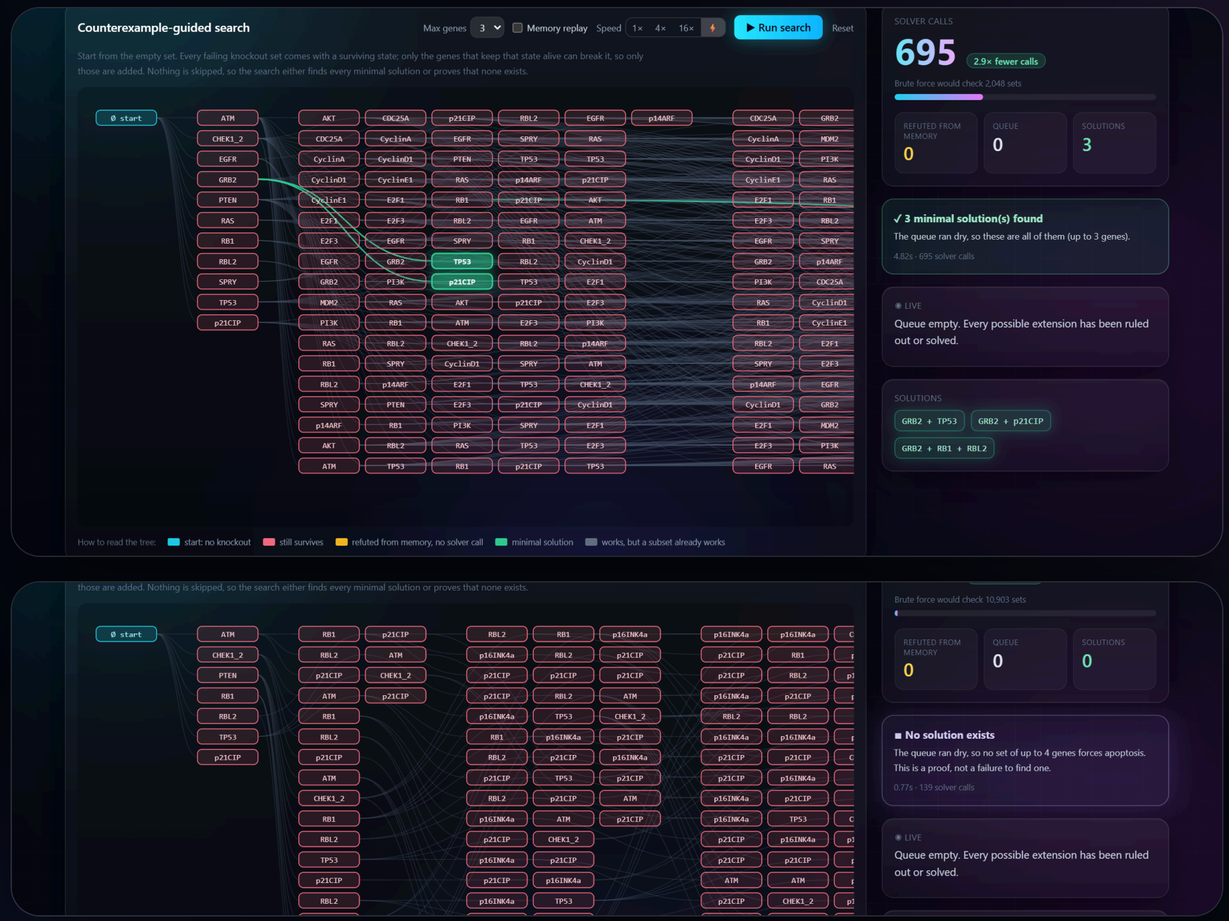
Nothing is skipped: every minimal solution is found, or the queue runs dry and **impossibility is proved**.

34×

fewer solver calls than brute force

56

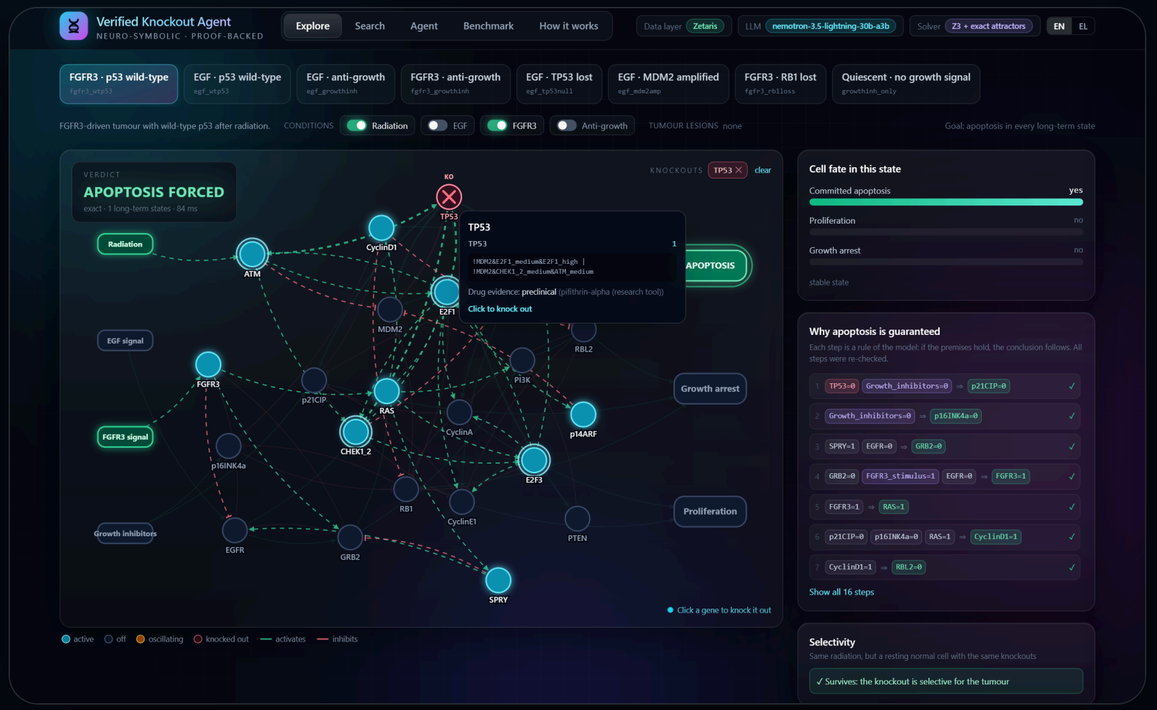
calls to prove “none up to 4 genes” (brute force: 10,903)



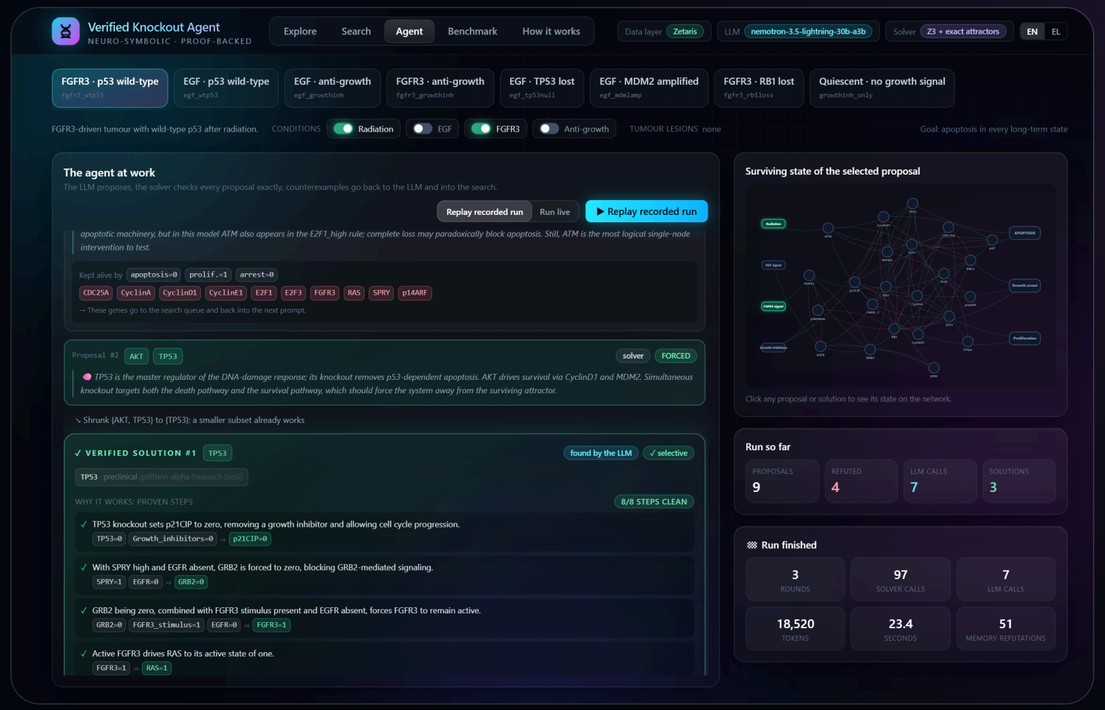
Search tree growing live in the UI · “No solution exists”: a proof, not a failure to find one

WHAT IT LOOKS LIKE

Click a gene and get an exact verdict with a proof you can read



Explore: knock out `TP53` and apoptosis is forced in every long-term state, with numbered proof steps.

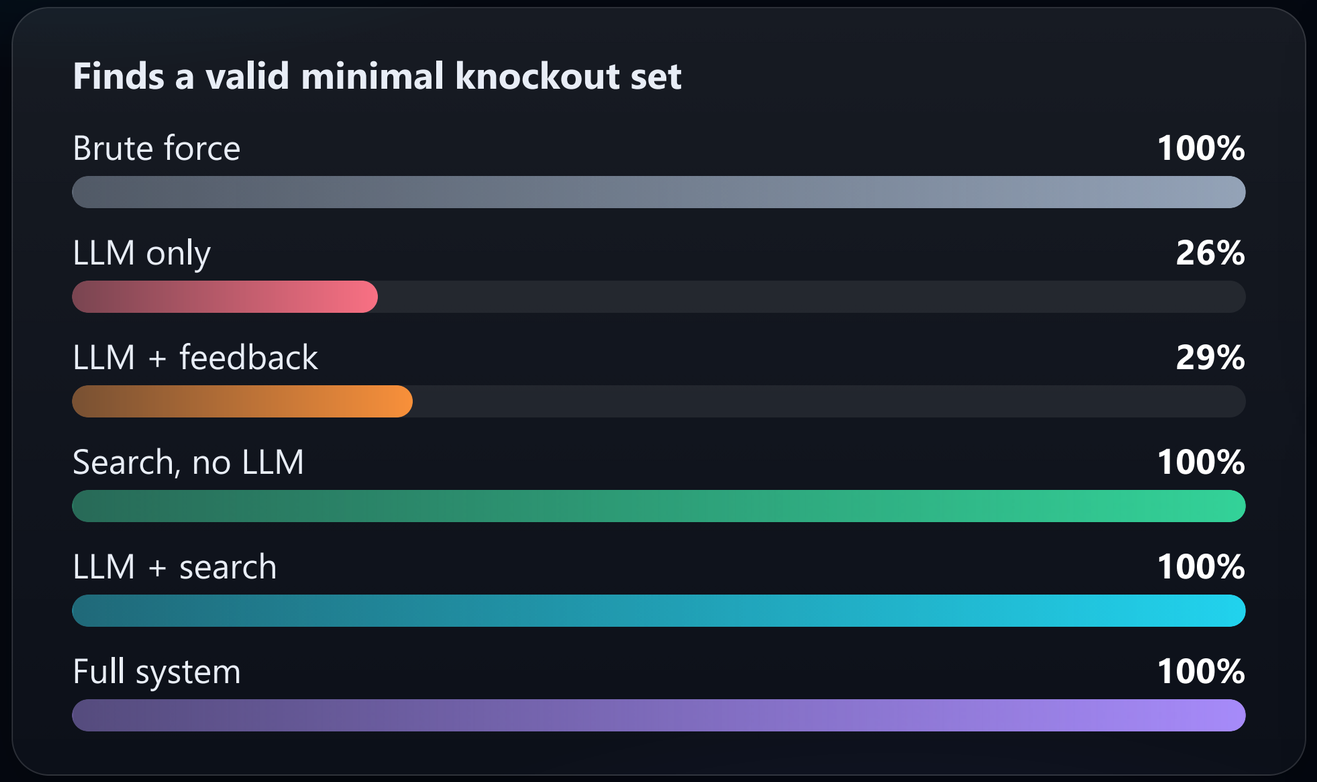


Agent: LLM proposals with rationale, counterexamples fed back, then a verified solution with checked steps.

Real runs recorded against Zetaris and Nemotron · replays work offline

MEASURED, NOT PROMISED

Never wrong with the solver, and honest about where the LLM helps



91%
LLM-alone proposals wrong

34×
fewer solver calls

13/13
official H-MEM conformance

97%
explanation steps stayed inside the proof

Honest reading: on this network the search is symbolic and needs no LLM. The LLM gives the interface and the wording, and every claim it makes is checked.

8 scenarios · exhaustive-solver ground truth · 3 repeats per LLM arm · 99 automated tests

SPONSOR TECHNOLOGY

Three sponsors, each doing real work in the loop

Zetaris

the data layer

Network rules, scenarios and drug evidence are 3 SQL tables in `open_agent`

The LLM writes its own `run_sql` queries; a guard allows one SELECT on our tables

A SQL JOIN ranks candidate genes. Benchmark answers are kept out of the tables

Meterless

Nightshift + H-MEM engine

Nightshift is the primary AI provider: every request goes through the local runtime, and the model that answered is logged

H-MEM, official and unmodified, is the memory: store, ranked recall into the next prompt, feedback, audit ledger

Test: 12 tests, conformance 13/13

NVIDIA Nemotron

the model, reached through Nightshift

`nemotron-3.5-lightning-30b-a3b` proposes knockouts, writes SQL, words explanations

The NVIDIA key stays in Nightshift, never in the project

Measured: thinking on 268 s, off 2 s. The 550B model words better (99.4% vs 94.6% clean) but proposes no better

What is unique: nobody in the loop is trusted. The model writes SQL but a guard validates it, proposes knockouts but an exact solver judges them, and the memory keeps only solver-verified facts.

TECHNICAL STACK

Exact reasoning at the core, sponsor services around it

Solver

`Python` `Z3` `numpy · scipy`

`asynchronous attractors (terminal SCCs)`

`counterexample-guided search`

Agent

`Nightshift /v1 (OpenAI-compatible)` `Nemotron`

`Zetaris REST + SQL` `H-MEM (Node bridge)`

Interface

`stdlib HTTP server + SSE` `vanilla JS modules`

`Tailwind (vendored)` `SVG network` `EN / EL`

Quality

`pytest, 99 tests` `exhaustive ground truth`

`reproducible benchmark` `SAMPLE_MODE offline`

Built in the window

Solver, search, explanation checker, agent, SQL tool, Nightshift client, memory integration, UI, benchmark, all tests

Pre-existing, declared

Remy et al. 2015 network · official H-MEM reference (Apache-2.0) · open-source libraries

Runs from a clean clone: **SAMPLE_MODE=true python ui/server.py** needs no account, key or service

LIMITS AND FUTURE ENHANCEMENTS

Upfront about the limits, clear about what comes next

Honest limits

In-silico hypotheses on **one** published model: not medical advice, expert and lab review required

Solutions are loss of tumour suppressors: no drug. No DNA-damage-response target appears, so the model misses mechanisms

Exact engine: about 21 free nodes. A 53-node model returned `UNKNOWN`

On this network the search needs no LLM: the LLM adds interface, SQL and checked wording

Future enhancements

Symbolic (BDD/SAT) attractor backend with an engine router, for larger networks

Validate against published knockouts and CRISPR screens

Tumour mutation profile becomes the scenario; druggable-only search

More Meterless engines (World Model, Markovian) and other models behind the same solver

The takeaway: a system that proves what it claims, and says “impossible” when it is.